Lab Report #4
Tyler Labrecque
James Colton Davis
10/01/25

Part A:

1. Block Diagram describing the process of generating pulsatile waveform:

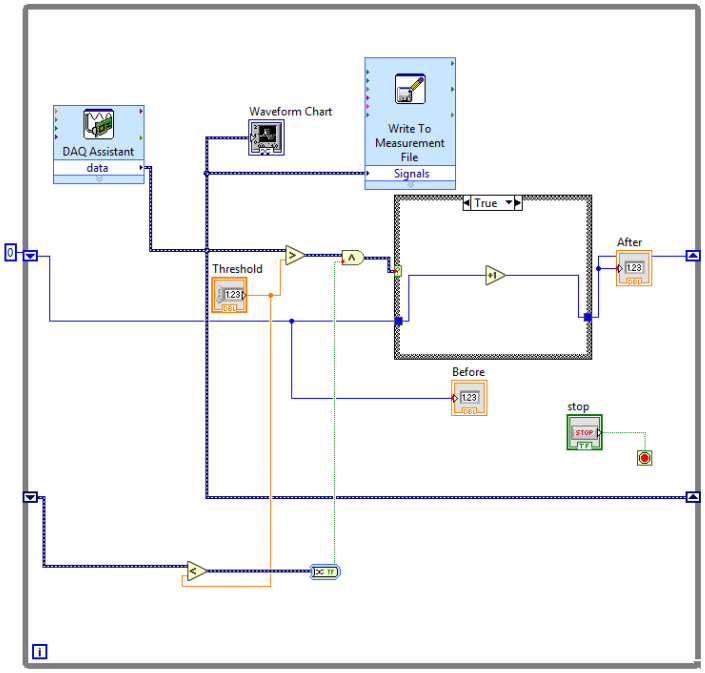

Figure 01: The corresponding figure represents the block diagram used to generate a pulsatile waveform through the use of a physical photoresistor. The figure shows how data from the DAQ assistant in connection with the waveform chart, is put through a case structure that will count pulses when the scenario is 'True', with 'True' indicating that the photoresistor was being covered by the group's hands. The shift resistor in the figure is used to compare the count value to the previous one, due to the fact that the number of counts generated can often be more than the actual amount of pulses. In short, the shift registers are connected to a Boolean tool and will form an additional count when n+1Boolean is greater than the n Boolean.

For the initial generating of the pulsatile waveform, it is generated simply by putting our finger/hand over the photoresistor which varies the light level. This varies the resistance and based on the block diagram if the voltage is less than the maximum value and greater than the minimum value that is attached to the case structure and shift registers, this will generate the

pulsatile waveform. The threshold voltage is determined and set to reduce signal noise. Data collected from the DAQ Assistant is displayed on the waveform chart and passed into a case structure that begins counting when the condition is True. In this setup, True indicates that the photoresistor is being covered by a hand or finger. This is not a true physiological pulse sensor like those used in clinical applications, however it does a good job at demonstrating how waveforms can be created, detected, and processed with LabVIEW given a pulsing input.

2. Circuit diagram (picture of your circuit or hand drawn) :

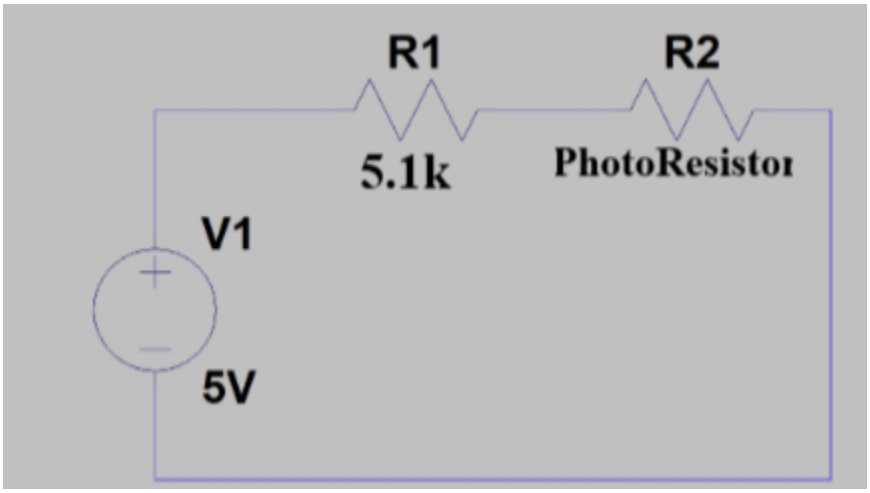

Figure 02: The circuit diagram of the simple voltage divider used, where the voltage is being read between the photoresistor and 1kOhm resistor. This figure clearly depicts that the 5.1kΩ resistor and the photoresistor provided in class are placed in parallel, whilst being supplied 5V from the myDAQ. This circuit diagram was the foundation for the circuit built/used in class.

3. Voltage Range, Resistor value and threshold voltage:

The threshold was held constant for all resistor tests and readings, at a value of 0. The following notates which resistor values were used/tested, and their resulting voltage ranges. For a resistor value of 15kΩ, the resulting voltage range was a low of 0.5V and a high of 2V. For the 10kΩ, 20kΩ, 1.0kΩ, 5.1kΩ, and 3.0kΩ, the corresponding voltage ranges were 1-3V, 0.4-0.7V, 2.5-5V, 0.8-3V, and 1.25-3V respectively. The 5.1kΩ resistor results in the most ideal voltage range as the low value is within the desired range of 0.0V to 0.8V, and the high value was properly within 2V to 5V with a value of 3V.

Resistor Values & Voltage Ranges:

- 15kΩ: 0.5 - 2V
- 10kΩ: 1-3V
- 20kΩ: 0.4 - 1.7V
- 1.0kΩ: 2.5 - 5V

- 5.1kΩ: 0.8-3V
- 3.0kΩ: 1.25 - 3V

Part B:

1. Block Diagram describing the process of generating and counting pulsatile waveform:.

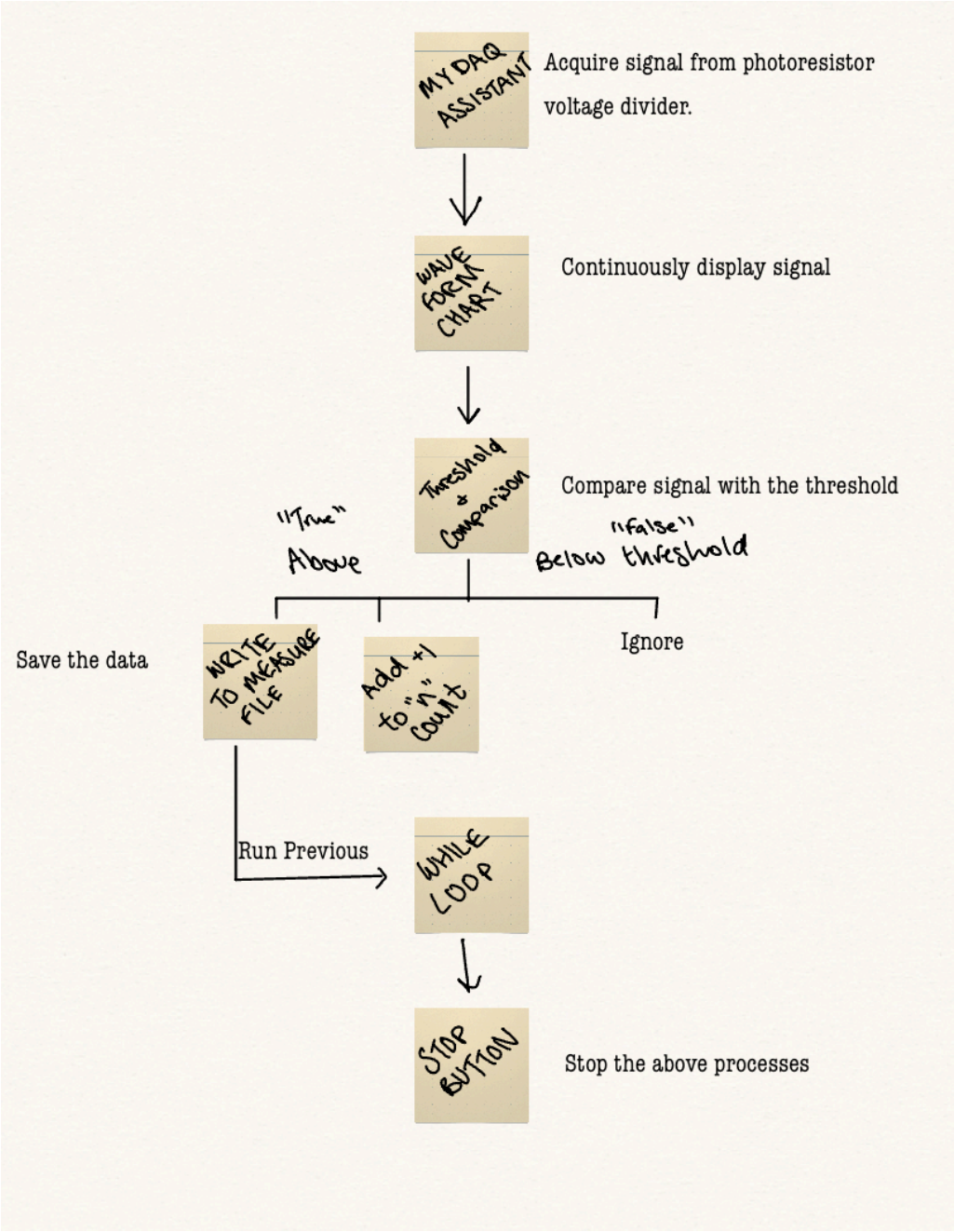

Figure 03: The figure above depicts a block diagram that simplifies the process of generating and counting a pulsatile waveform. This figure helps give the same process that is constructed in

the following LabVIEW panels a more linear, and easier to understand pathway. The myDAQ assistant within the LabVIEW software acquires a signal from the photoresistor from the physical in-lab build, which is continuously displayed/monitored on the waveform chart, the signal itself is then compared with the threshold that has been set by the team. The boolean nature of the build then denotes the signal as 'False' if it is below the threshold, in which case it is ignored, or 'True' if it is above/in the threshold at which point the data is saved via the write to measurement file tool and the looping process continues uninterrupted.

2. LabVIEW VI Images + description of the process of counting pulses.

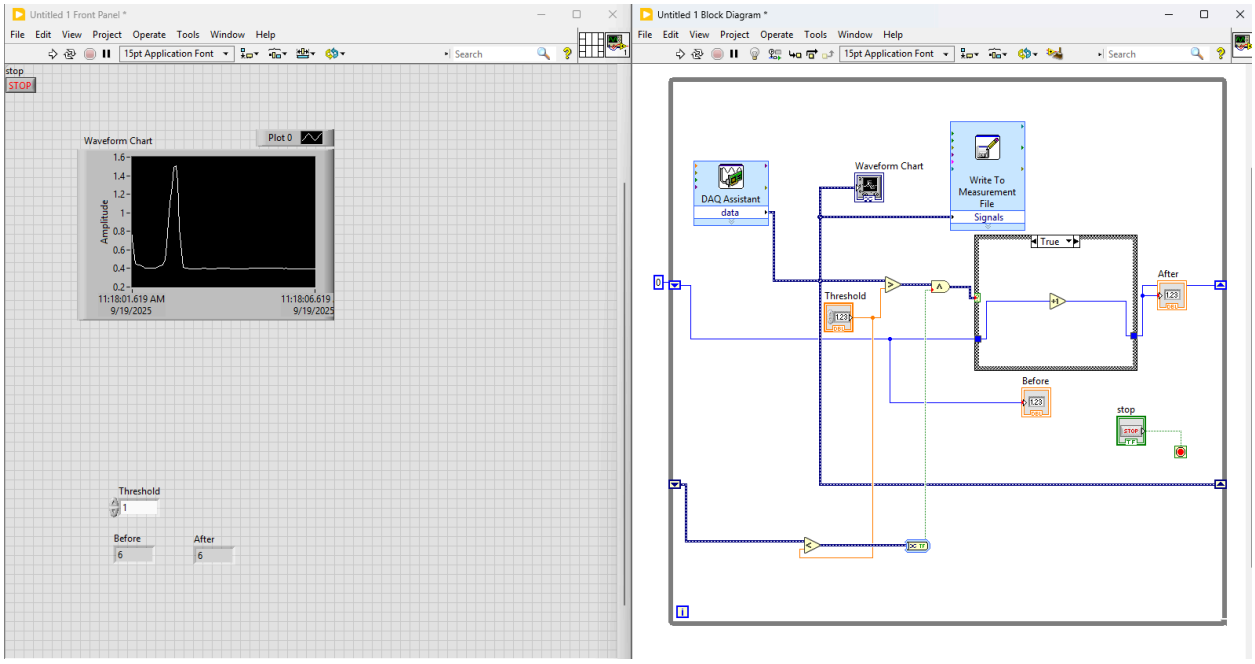

Figure 04: This figure is representative of the actual software that was constructed within LabVIEW. The build above follows the same processes described by Figure 03. Here we can see how the block diagram correlates with the placement of virtual wires and tools. The MyDAQ assistant can be seen displaying the signal to the waveform chart above while also meeting the inequality requirements of the threshold blocker. Past this point the signal must pass or be rejected through the case structure before adding a count to the pulse or being stored by the write to measurement file.

To avoid overcounting, the shift register is implemented to compare the current count value with the previous one. Because the signal can sometimes generate multiple counts for a single pulse, the shift register ensures that an additional count is only added when the Boolean value at time n+1 is greater than at time n. This logic detects the leading edge of each pulse. As shown in class, the block diagram works by adding 1 to the count at the beginning of the pulse (after it surpasses the threshold value) and then considered the instance complete after the pulse is lower than the threshold value (on the negative slope of the pulse). Thus each pulse is only counted

once. This value is vital for the next part as this is “number of pulses” used in the BPM calculation equation.

3. Excel plot of data showing the pulsatile waveform. (Label the waveform with threshold voltage line)

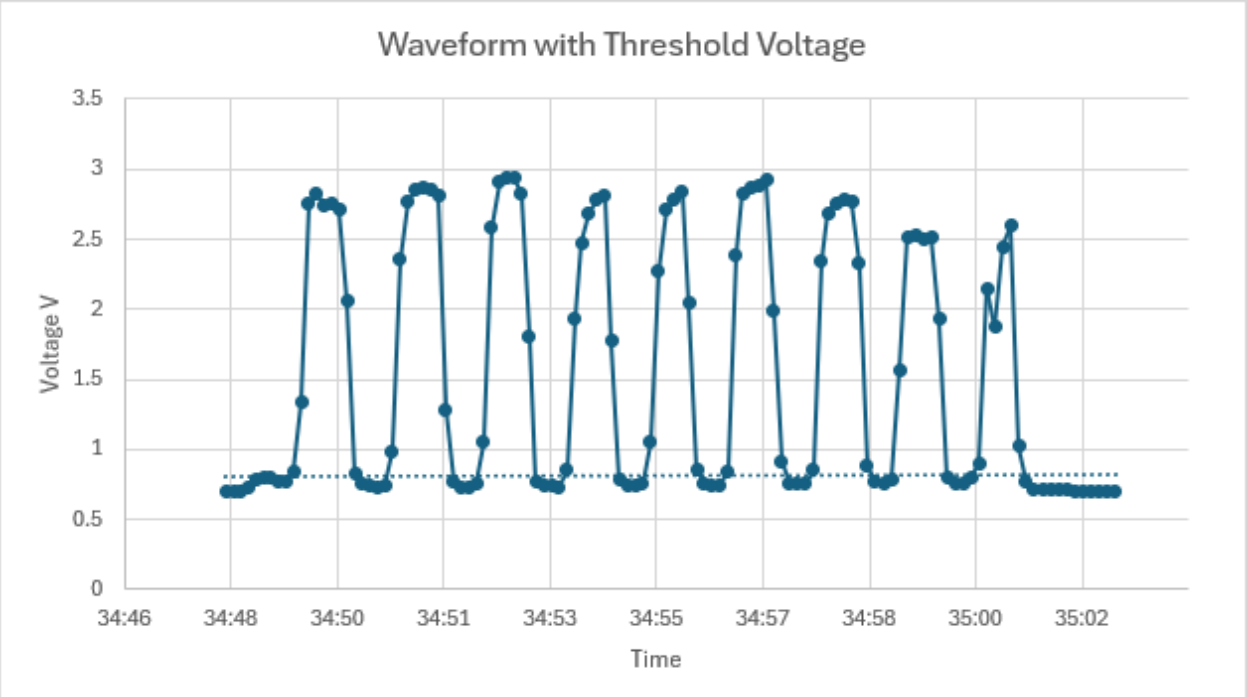

Figure 05: The pulsatile waveform that was recorded by the write to measurement file tool was saved and plotted as seen in this figure. The time is incremented by seconds in relation to the change in voltage readings from the pulse of the photoresistor being manipulated on the physical circuit build. A trend line was applied to the chart which is indicative of the threshold line which is backed up by our experimental value of about 0.80V, as denoted by the dotted line. This chart clearly shows both pulse, and how the signal rests above the threshold set by the team.

Part C:

1. Block Diagram describing the process of obtaining BPM from Digital Input Line.

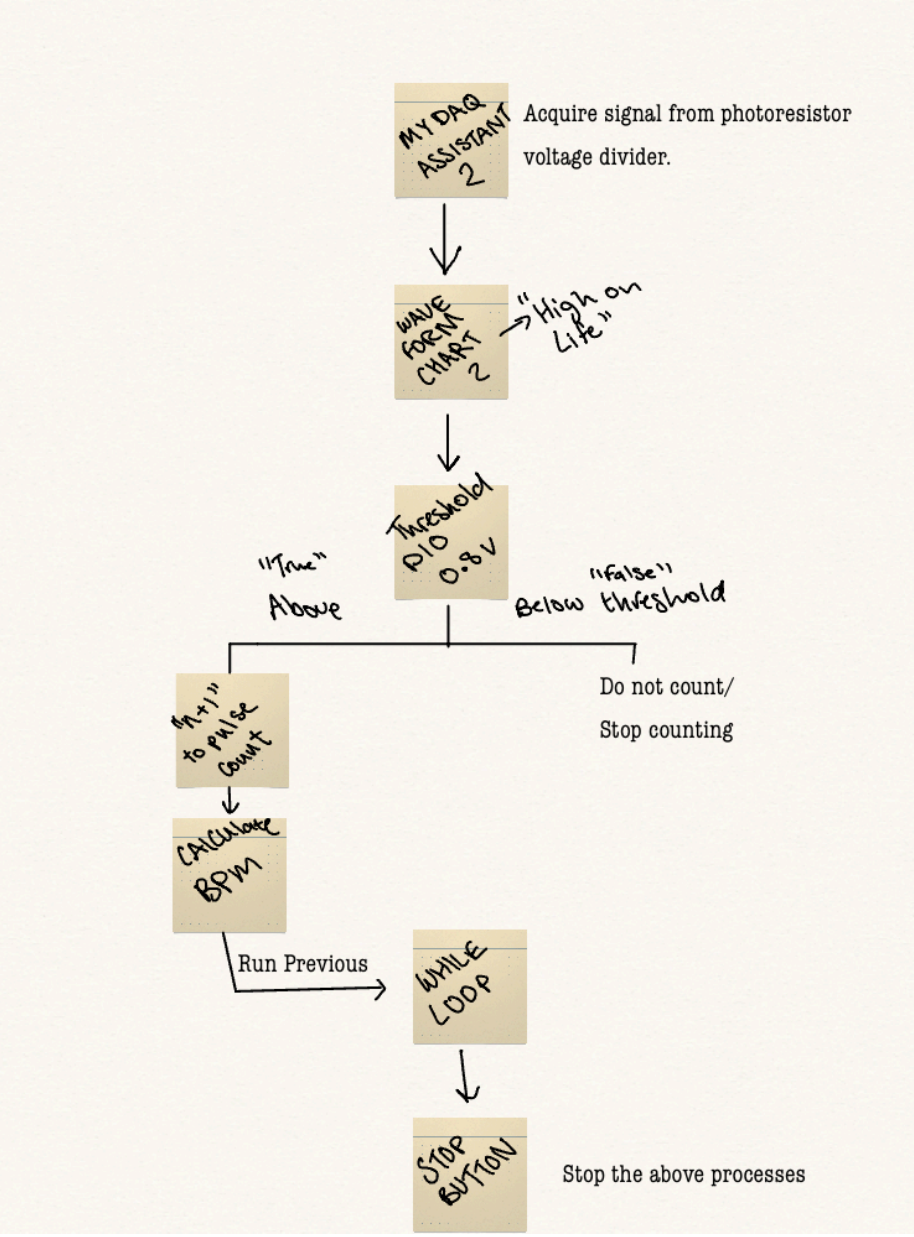

Figure 06: The figure depicts the process of obtaining BPM and the digital input line in a more linear format. The block diagram, much like Figure 03, maps out the logistical workings of the LabVIEW software for Part 3 of the lab. Similarly, the process begins with the acquisition of the signal from the photoresistor voltage divider with help of the MyDAQ assistant, at which point the signal is presented on a different waveform chart than the previous one. Next is a threshold requirement that was valued at 0.8V, if the signal's value was above the set value, it passed under 'True' and an additional value would be added to the pulse count and run through the BPM calculation before continuing the loop. On the other hand, if the value of the signal was below the threshold, no count would be added, and the counting as a whole would be stopped.

2. LabVIEW VI Images + description of the process of calculating BPM.

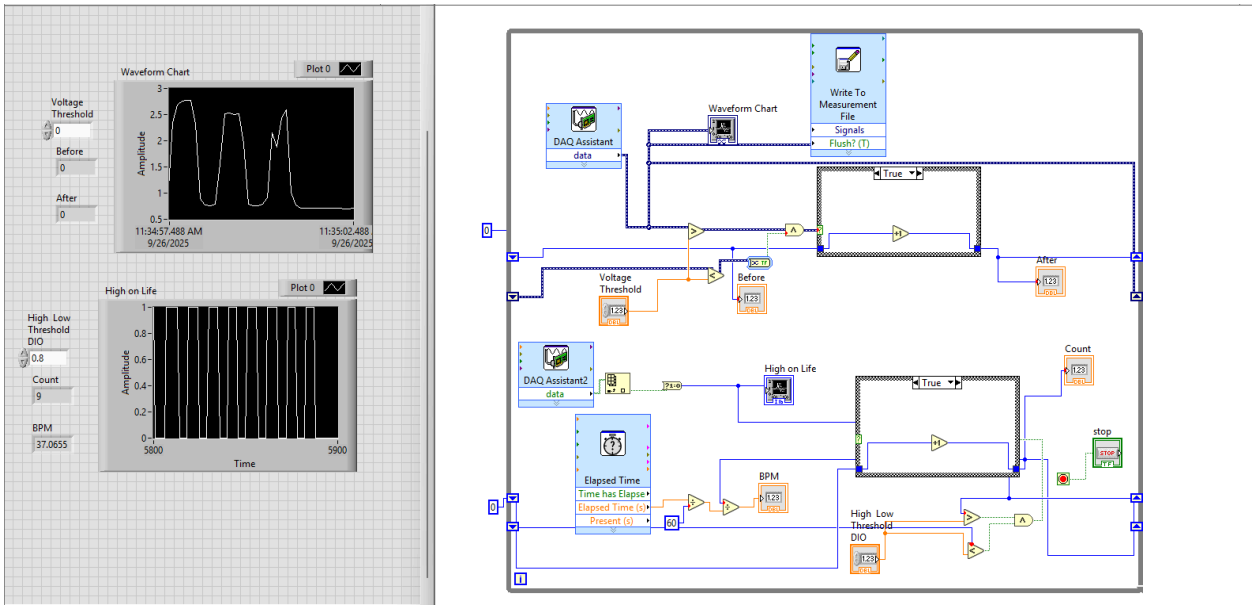

Figure 07: The front panel and the block diagram from the LabVIEW software for the BPM calculation and data collection can be seen in this figure. The new data collection process can be seen added under the one form parts 1-2, indicated by a second MyDAQ Assistant which obtains data from the digital input line of the myDAQ. The second waveform chart can also be seen on the bottom right hand side of the figure. The new software build matches the logic steps described by Figure 06, however using the digital input is important as it reads the binary states of the signal, aiding the precision of the logic and making a concrete 0.8V threshold.. The figure depicts the signal as it moves from the DAQ Assistant through the waveform chart, a case structure, and how a true value will add a count, as well as take the signal with the elapsed time and numeric operator tools to calculate the BPM.

The process of calculating Beats Per Minute (BPM) begins by acquiring a pulsatile signal from the photo-resistor circuit using the myDAQ and LabVIew interface. This signal reflects a change in the light intensity of the physical photoresistor as manipulated by the group's hands, which is meant to represent the hypothetical bloodflow. In LabVIEW, the analog waveform is continuously monitored, and a threshold comparison is applied to detect when a waveform crosses a preset level, then converting it into a boolean signal that can mark each hypothetical heartbeat. A shift register within the loop stores the previous signal value and compares it to the current value being read. After this, a case structure helps to increment the pulse counter, but only when the signal transitions from below the threshold to above it, which helps the group ensure each beat is only counted once. The number of pulses counted over a defined time interval is then used to calculate BPM with formula (1).

$$\text{BPM} = \frac{Number\ of\ Pulses}{Time\ Interval\ (s)} \times 60 \quad (1)$$

The BPM value is calculated thanks to the use of the numeric operators in LabVIEW. For the division operators, the x (top) input is the numerator of the operation, where the y (bottom) input is the denominator and the output is pushed out the front of the arrow. Starting from the beginning, we can see the following logic by tracing the numerators and denominators.

$$\text{BPM} = \frac{Case\ Structure\ (Number\ of\ Pulses)}{\frac{Elapsed\ Time(s)}{60}} \quad (2)$$

The first division operator helps to calculate the relationship between the factor of 60 and the elapsed time which is then moved into another division operator that is also attached to the case structure, ultimately outputting a BPM value. Obviously, the equation above gets simplified to the following:

$$\text{BPM} = \frac{Number\ of\ Pulses}{Elapsed\ Time\ (s)} \times 60 \quad (3)$$

Thus the BPM is calculated and finally, a waveform chart displays both the original analog signal and the processed pulses, while numeric indicators show the calculated BPM for continuous monitoring.